

الملحق التقني الأخير – الوثيقة الرابعة والختامية

AI Memory System · AI Prompting Strategy · Offline Strategy · Content Management (CMS-Ready)

بإضافة هذه الوثيقة، تُعتبر حزمة التصميم الكاملة (4 وثائق) جاهزة بنسبة 100% لبدء التنفيذ.

1. AI Memory System

1.1 القرار التصميمي

طويلة تُعاد إرسالها كاملة في كل مرة (مكلفة وغير AI الذاكرة) ليست محادثة " **Structured User Memory Profile** موثوقة)، بل ملف مستخدم منظم AI. يُبنى ويُحدَّث باستمرار من قاعدة البيانات، ويُمرَّر كسياق مضغوط لكل استدعاء.

التبرير: الاعتماد على "سجل محادثة كامل" يُعاد إرساله للنموذج في كل مرة يكبر بسرعة (تكلفة + بطء)، ولا يضمن أن النموذج "يتذكر" التفاصيل المهمة تحديدًا. بناء ملف منظم يعني: نحن نقرر بدقة ماذا يتذكره "نبيه"، لا نترك الأمر لقدرة النموذج العشوائية على الاستدعاء من نص طويل.

1.2 يُبنى – (UserMemoryProfile) محتوى الذاكرة ديناميكيًا من الجداول الموجودة

الفئة	المصدر (جدول)	AI مثال قيمة فعلية تُمرَّر للـ
الهوية التعليمية	Users, UserProfile	المستوى الحالي، الهدف (عمل/سفر/...)، تاريخ

الفئة	المصدر (جدول)	AI مثال قيمة فعلية تُمرَّر للـ
		الانضمام
قاموس نشط	Words + WordReviews (آخر 20 كلمة، مرتبة بحسب lifecycleStage)	الكلمات الخمس الأخيرة قيد "invoice, deadline, meeting..."
أنماط الأخطاء المتكررة	MistakePatterns (غير المحلولة, occurrences ≥ 2)	"يخطئ بشكل متكرر في" Past Simple (5 مرات)، "حروف الجر (3 مرات)"
سياق الجلسة الحالية	الدرس/الوحدة النشطة الآن	"الوحدة الحالية": Work English, الكلمات المستهدفة schedule, اليوم colleague"
تفضيل الأسلوب	Settings (لاحقًا)	سرعة النطق، طول الشرح المفضل (قصير/مفضل)
آخر تفاعل	DailyJournal (آخر سجل)	آخر جلسة: أمس، أتقن كلمات "التحية، تعثر في بناء الأسئلة"
حالة المسار	LevelExams (إن وجدت) (محاولة سابقة)	A0: محاولة سابقة لامتحان Writing رسب في قسم "تحديدًا"

آلية البناء التقنية 1.3

دالة واحدة مركزية `buildUserMemorySnapshot(userId, purpose)` في طبقة `core/ai/`:

- مثلًا استدعاء `(purpose)` تستعلم فقط عن الحقول ذات الصلة بالغرض "تصحيح جملة" لا يحتاج تاريخ الامتحانات الكامل، بينما "توليد تقرير يومي" يحتاجه.
- Prompt مضغوط (لا نص حر طويل) يُمرَّر لاحقًا إلى JSON تُعيد كائن Builder (البند 2).

- الإيجاز – (token) رمز ~500 snapshot حد أقصى للحجم: لا يتجاوز ال إلزامي، فالذاكرة الجيدة تُلخّص لا تُفرغ كل البيانات

1.4 التحديث المستمر

كل تفاعل مهم (خطأ متكرر جديد، كلمة أتقنت، محادثة اكتملت) يُحدّث الجداول دائماً "طازج" في Snapshot المصدريّة فوراً (لا تجميع مؤجل)، بحيث ال "الاستدعاء التالي – هذا يحقق شعور "يتذكرني فوراً" لا "يتذكرني بعد فترة

2. AI Prompting Strategy

2.1 القرار التصميمي

عام واحد Prompt (لا منفصل ومستقل AI Prompt Template لكل وظيفة مُختبر ومضبوط لغرضه فقط، عبر template كل – "ذكي" يحاول فعل كل شيء `AITeacherGateway` واجهة موحدة

واحد شامل لكل الوظائف يصعب اختباره وضبطه (كل تعديل قد Prompt: التبرير يكسر وظيفة أخرى غير مقصودة)، ويصعب التحكم بالتكلفة (بعض الوظائف تحتاج سياقاً غنياً، أخرى بسيطة جداً ولا يجب إثقالها). الفصل يعني كل وظيفة تُقاس وتُحسّن بمعزل عن الأخرى

2.2 Prompt Templates قائمة ال

المعرّف	الوظيفة	مدخلات الذاكرة المطلوبة (من البند 1)	ة على المخرجات
<code>sentence.correct</code>	تصحيح جملة	الهدوية التعليمية + المستوى فقط	JSON: {i correctedTe explanation matchedGram

المعزف	الوظيفة	مدخلات الذاكرة المطلوبة (من البند 1)	ة على المخرجات
word.explain	شرح كلمة / توليد أمثلة	المستوى + قاموس مصرّح به (allowed vocabulary pool)	فقط كلمات ضمن لمستخدم أو أدنى
exercise.generate	توليد تمرين	مفردات الوحدة الحالية فقط	يُخزّن في كاش AIGeneratedC، يُعاد استخدامه
conversation.turn	رد AI أثناء محادثة	الهدف + الوحدة الحالية + أنماط الأخطاء المتكررة	دام مفردات أعلى طول رد ≥ 3 جمل
journal.dailyReport	كتابة التقرير اليومي	ملخص جلسة اليوم + أنماط الأخطاء + آخر جلسة	نزم حرفيًا بقواعد Voice (البند 12)
grammar.detect	اكتشاف نمط خطأ نحوي	نص الجملة الخاطئة فقط	يُعيد فقط matchedGran محددة مسبقًا (لا جديدة من عنده)

المعزف	الوظيفة	مدخلات الذاكرة المطلوبة (من البند 1)	مخرجات
<code>exam.evaluateWriting / exam.evaluateConversation</code>	تقييم أقسام الامتحان	نص/محادثة المستخدم + معايير المستوى	{score, strengths[], weaknesses[]} يُعيد قابلة للتخزين في LevelExams

Prompts قواعد صارمة عابرة لكل ال 2.3 (Guardrails)

1. **System Context** ثابت في كل استدعاء: "أنت نبيه، معلم إنجليزية. التزم حصراً بمفردات المستوى وما دونه. لا تتجاوز طول الرد [X] شخصي. التزم حصراً بمفردات المستوى لا يغير AI" (المحدد) – هذا يفرض تقنياً القاعدة المذكورة في وثيقة المنهج ("المنهج").
2. `conversation.turn` منظمة إلزامياً لكل الوظائف عدا `JSON` مخرجات هذا – (نص حر لكن بقيود طول وأسلوب) `journal.dailyReport` ويسهل التحقق الآلي ومنع كسر الواجهة عند استجابة غير متوقعة.
3. **Fallback** إن فشل الاستدعاء أو تجاوز الوقت المحدد: **إلزامي لكل Prompt** يُعرض رد احتياطي معد مسبقاً (مثال: "لم أتمكن من التحقق الآن، (timeout) حاول مجدداً بعد قليل") بدل تعليق الواجهة أو عرض خطأ تقني للمستخدم.
4. كل استجابة (بدون بيانات شخصية): **تسجيل جودة (Eval Logging)** حساسة) تُسجل مجهولة المصدر لعينة صغيرة لمراجعة دورية من فريق **Prompts** المحتوى – أساس أي تحسين مستقبلي للـ

3. Offline Strategy

3.1 القرار التصميمي

المحلي هو مصدر الحقيقة دائماً، والاتصال SQLite: حقيقي **Offline-First** مبدأً حياً. أي شيء آخر يعمل بلا إنترنت AI بالإنترنت مطلوب فقط للوظائف التي تتطلب بالكامل.

يعمل بلا إنترنت (100%) 3.2

- تصفح المنهج، الكلمات، الأمثلة، الوحدات (كلها محلية منذ التثبيت أو أول مزامنة).
- القراءة والتحديث محلياً بالكامل – SRS (Flashcards) مراجعة.
- Statistics, My Dictionary, السابق Learning Journal عرض.
- لا يحتاج (Expo Speech) عبر محرك النظام المحلي – (TTS) النطق الصوتي اتصالاً.
- تُحفظ محلياً بحالة "بانتظار" Sentence Builder تسجيل جملة في ("التصحيح").

(حي فقط AI) يحتاج اتصالاً 3.3

- (sentence.correct) تصحيح جملة فعلي.
- (conversation.turn) AI محادثة.
- إن لم يتوفر اتصال عند – (journal.dailyReport) توليد تقرير يومي.
نهاية اليوم، يُبنى تلقائياً عند أول اتصال متاح لاحقاً (لا يضع اليوم).
- تقييم أقسام الامتحان الكتابية/المحادثة.

(Sync Queue) آلية قائمة الانتظار 3.4

- يحدث بلا اتصال يُحفظ في جدول محلي AI أي طلب
`PendingAIRequests (id, type, payload, createdAt, status)`.
- يُنفذ الطلبات (background sync) عند استعادة الاتصال، معالج خلفي
المعلقة بالترتيب، ويُحدّث الواجهة فوراً عند اكتمال كل واحد (إشعار داخلي خفيف: "تم تصحيح جملتك من أمس").

- حالة خاصة بالجمل: إن كتب المستخدم عدة جمل بلا إنترنت، تُصَحَّح كلها دفعة واحدة عند العودة، لا واحدة تلو الأخرى بشكل مربك.

3.5 تعارض المزامنة (Conflict Resolution)

بلا حسابات سحابية (بيانات محلية فقط)، لا يوجد MVP/V1 بما أن التطبيق في (V2) كما في هذه المرحلة. عند تفعيل المزامنة السحابية **multi-device** تعارض على (Last-Write-Wins) "تُعتمد سياسة" آخر تحديث يفوز، (في خارطة الطريق `stageHistory`: مستوى الحقل الواحد لا السجل الكامل، مع استثناء لا يُستبدل، لأنه سجل تاريخي (merge) يُدمج (سجل تراكمي) WordReviews ولا يجوز فقده.

3.6 تجربة المستخدم أثناء انقطاع الاتصال

فقط (لا رسالة AI مؤشر هادئ وغير مقلق في الزوايا العلوية للشاشات التي تحتاج أنت غير متصل الآن، "Tone of Voice: خطأ حمراء صارخة) – نص متسق مع "سأصحح جملتك فور عودة الاتصال" بدل "خطأ في الشبكة".

4. Content Management (CMS-Ready)

4.1 القرار التصميمي

المحتوى المنهجي

(Levels/Modules/Words/Phrases/GrammarRules/Exam

يُصمَّم من اليوم الأول كبيانات منظَّمة قابلة للتحريك خارج الكود، (questions بحيث بناء لوحة إدارة لاحقاً هو – MVP حتى لو لم توجد لوحة تحكم فعلية في عمل واجهة فقط، لا إعادة هيكلة قاعدة بيانات

فكل إضافة، (ثابتة seed files) التبرير: لو بقي المحتوى مكتوباً داخل ملفات كود كلمة أو تعديل وحدة يتطلب إصدار تحديث تطبيق جديد – عنق زجاجة تشغيلي

خطير لمنتج محتواه هو صلب قيمته. الفصل من البداية يعني فريق المحتوى (غير المبرمجين) يمكنهم لاحقًا التعديل مباشرة.

4.2 مبادئ تصميم البيانات القابلة للإدارة

1. يسمح — `status (draft / published / archived)` كل كيان محتوى له لاحقًا بمراجعة محتوى جديد قبل نشره للمستخدمين دون التأثير على من يستخدم التطبيق حاليًا.
2. تتبع — `updatedBy` و `updatedAt` و `version` كل كيان محتوى له تغييرات كامل، ضروري لأي لوحة تحكم مستقبلية.
3. داخل كود الميزات. كل نص تعليمي (شرح `Hardcoded` لا نصوص أو قواعد قاعدة، سؤال اختبار، مفتاح إجابة) يُقرأ من قاعدة البيانات حصريًا.
4. لا رقم (UUID) لكل كلمة/وحدة/قاعدة (Stable IDs) معزفات مستقرة. لا يضمن أن التعديل اللاحق عبر لوحة تحكم لا يكسر — (تسلسلي قابل للتغيير (WordReviews, MistakePatterns, إلخ) الروابط.
5. فصل "بنية المنهج" عن "محتوى اللغة" (كما في وثيقة قابلية التعدد اللغوي). كهيكل موحد، وتدير Modules/Levels لوحة التحكم تدير CMS: يخدم أيضًا كمحتوى منفصل يمكن تكراره لكل لغة مستهدفة دون تكرار Words/Phrases الهيكل.

4.3 مستقبلًا (بلا تعديل CMS الجداول الجاهزة لإدارة بنيوي)

جميع الجداول التالية مؤهلة بالفعل من التصميم السابق لثدار عبر لوحة تحكم مباشرة، بإضافة الحقول الإدارية فقط (البند 4.2):

بنك (Levels, Modules, Words, GrammarRules, LevelExams (مستقل ExamQuestions الأسئلة إن فصل لاحقًا كجدول

مستقبلًا دون تعقيد الآن CMS إضافة مقترحة جديدة لدعم

| جدول جديد | الغرض |

|---|

| سجل كل تعديل على أي محتوى (من سيضيفه لاحقًا عبر | ContentAuditLog

(entityType, entityId, changeType, changedBy, changedAt, diff (JSON) |

بنك أسئلة قابل لإعادة | LevelExams فصل عن ExamQuestions |
الاستخدام والتوليد العشوائي لكل محاولة امتحان، بدل أسئلة ثابتة مكررة تُحفظ أو
| تُنسى بسهولة

خطة لوحة التحكم المستقبلية (خارج نطاق 4.4 البرمجة الحالية، للعلم فقط)

لاحقًا (على الأرجح تطبيق ويب منفصل بسيط)، سيكون عمله: CMS عندما يُبنى
V2 أو نسخة سحابية منها بعد SQLite قراءة/كتابة مباشرة على نفس جداول
بدون أي منطق عمل إضافي مطلوب من التطبيق نفسه — لأن كل قواعد التحقق
(مثال: كلمة يجب أن تنتمي لمستوى موجود) موجودة أصلًا على مستوى
Schema (Foreign Keys + Constraints)، لا داخل كود الشاشات.

5. اعتماد نهائي

بإضافة هذه الوثيقة إلى الحزمة، التصميم الكامل يتضمن الآن أربع وثائق متكاملة:

1. **architecture-plan / product-design-full / first-week-ux-and-mvp** — PRD, SDD, DB أساسي UX، نطاق MVP الأسبوع الأول.
2. **master-design-v2** — Grammar on المنهج، دورة حياة الكلمة — PM. مراجعة، Memory Map، الامتحان النهائي، Demand.
3. **brand-identity-design-system** — الهوية، نظام التصميم، شخصية — Tone of Voice، المعلم.
4. العمل بلا إنترنت، جاهزية، Prompts استراتيجية الـ AI، هذه الوثيقة — ذاكرة CMS.

.لا فجوات تصميمية معلقة تمنع بدء الكود

الخطوة التالية الفعلية

بناء المشروع تقنيا بالترتيب التالي:

1. Expo + TypeScript + Expo Router + المجلدات (Clean Architecture). تهيئة
 2. Schema + (كل الجداول من الوثائق الأربع) SQLite كامل لقاعدة بيانات migrations.
 3. Design System tokens ومكوناته الأساسية.
 4. أول + buildUserMemorySnapshot + AITeacherGateway Prompt template (sentence.correct) كإثبات مفهوم (proof of concept) قبل الباقي.
 5. التنسج بالترتيب المتفق عليه MVP شاشات.
- هل نبدأ الآن بالخطوة 1 (تهيئة المشروع)؟